

Rapport – Devoir 1 – IFT2035

Groupe 67

1. Allocateur à pile :

Pour la première partie du devoir, l'allocateur à pile, on devait seulement modifier la méthode `.alloc` de la struct `AllocateurPile`. Pour ceci, il a fallu :

- Récupérer un pointeur vers l'instance de notre allocateur (`AllocateurPile`) qui est stocké dans `self`
- Calculer l'alignement en octets
- Trouver l'adresse de base du buffer de notre allocateur (`base_addr`)
- Trouver la prochaine adresse « alignée »
 - On additionne `base_addr` avec `self.next` pour trouver où la prochaine allocation pourrait se faire
 - On utilise `std.mem.alignForward` pour arrondir au prochain multiple de l'alignement
 - On converti l'adresse alignée à un offset du buffer de l'allocateur en faisant la soustraction avec `base_addr`
- On check le cas où on serait out of memory (si `position + len > longueur du buffer de l'allocateur`)
 - Si out of memory, return null
 - Sinon on avance le curseur de `self.next` de `len`

2. Etiquetage des allocations

Pour cette partie, on doit étiqueter les allocations faites à l'aide d'une struct `Header`, qui contient 2 attributs :

- `Len` = longueur de l'allocation
- `Free` = bool pour savoir si l'allocation en question a été libérée.

Pour ceci, on doit compléter 3 méthodes : `alloc`, `getHeader`, et `free`.

Pour `alloc`, conceptuellement, c'est assez similaire à ce qui a été réalisé durant l'allocateur à pile, sauf qu'on doit aussi :

- Trouver de l'espace et aligner le header

- Checker l'espace pour len + header size
- Trouver l'espace pour le payload en fonction de son alignement après le header

Pour le getHeader :

- On calcule l'adresse du header grâce au header size et payload adress
- On cherche et retourne un pointer de header de l'adresse qu'on a calculé avant

Pour le free :

- On obtient un pointeur vers le header (grâce à la méthode getHeader)
- Set l'attribut de free à true.

3. Recyclage

Pour pouvoir réutiliser les blocs de mémoires qui sont libres dans notre implémentation, on a dû modifier 3 méthodes : alloc, free, getHeader.

Pour alloc :

- On calcul la taille du header, base adresse (l'adresse de début du buffer) comme dans les implémentations précédentes.
- On itère sur les blocs existants pour trouver un bloc libre qui peut être réutilisé :
 - Pour ceci, on check si le header du bloc est libre (header.free = True) et que le bloc est assez grand pour notre allocation (header.len >= len)
 - Si on trouve un bloc qui satisfait toutes ces conditions, on le ré-utilise à la place de consommer un nouveau bloc.
 - Sinon, on se déplace pour voir les autres blocs
- Si aucun bloc ne peut être réutilisé, on en crée un à la fin.

Le free et getHeader restent les mêmes que celui de l'étiquetage.

Un bog qu'on a eu et qui était intéressant était sur l'alignement au niveau de alloc

std.mem.alignForward(usize, self.next, alignment.toByteUnits()); (avant) qui passait les test cases chez un co-équipier (Mac-OS) mais pas chez l'autre (Windows avec WSL).

Une solution a été de modifier le code à ceci :

```
const aligned_addr = std.mem.alignForward(usize, @intFromPtr(self.buffer.ptr); +
self.next, alignment.toByteUnits());
```

Ceci semble marcher pour nous deux.

Utilisation des LLMs :

- **Aide dans le debug des erreurs de Zig**